CLINIC APPOINTMENT REQUEST APP

Project Overview

Elevator Pitch

For clinics seeking to modernize their appointment booking, our app provides a seamless, user-friendly platform for patients to request appointments, reducing administrative burden and improving patient access.

Problem Statement

Patients often face inconvenient and time-consuming methods for requesting clinic appointments, leading to frustration and potential delays in care. Clinics struggle with manual scheduling processes, resulting in inefficiencies and potential errors.

Solutions

A simple web application allowing users to sign up, log in, and submit appointment requests with their preferred date, time, and a description of their problem, streamlining the process for both patients and clinic staff.

Success Metrics

Increase in online appointment requests by 30% within 6 months, reduction in phone call appointment requests by 20%, and 90% user satisfaction with the appointment request process.

Competitors

Zocdoc, MyChart, local clinic websites with online forms, phone-based appointment systems.

Vision

To become the most intuitive and efficient platform for patients to connect with healthcare providers for appointment scheduling, fostering better patient engagement and operational efficiency for clinics.

Target Audience

Patients

Needs & Goals: Easy, quick, and convenient way to request appointments; clear communication regarding appointment status; ability to provide detailed information about their problem.

Clinic Administrators/Staff

Needs & Goals: Streamlined process for managing appointment requests; clear overview of pending requests; efficient communication tools; reduced manual data entry and phone calls.

Key Features

User Authentication

Allows users to sign up and log in securely to access the appointment request form.

Appointment Request Form

A form for users to submit their name, contact number, preferred date and time, and a description of their problem.

Backend API for Request Submission

An API endpoint to securely receive and store appointment requests from the user-facing form.

Admin Dashboard for Request Management

A secure interface for clinic staff to view, manage, and respond to submitted appointment requests.

Phasing

Phase 1: MVP

Rationale: This phase includes all core functionalities required for users to sign up, request appointments, and for clinic staff to manage these requests. It represents a minimum viable product that can be launched to early users.

Features:

- User Authentication
- Appointment Request Form
- Backend API for Request Submission
- Admin Dashboard for Request Management

User Flows

User Signup and Login

1. User navigates to the app's login/signup page.
2. User clicks 'Sign Up' and enters email and password.
3. System creates a new user account and logs the user in.
4. Alternatively, user clicks 'Login' and enters existing credentials.
5. System authenticates user and redirects to the appointment request form.

Submit Appointment Request

1. User logs in to the app.
2. User sees the appointment request form.
3. User fills in name, number, preferred date, preferred time, and problem description.
4. User clicks 'Submit Request'.
5. System validates the input and sends the request to the backend.
6. System displays a confirmation message to the user.

Clinic Staff Views and Manages Requests

1. Clinic staff logs into the Admin Dashboard.
2. Staff sees a list of pending appointment requests.
3. Staff can view details of each request (patient name, contact, preferred time, problem).
4. Staff can mark requests as 'reviewed', 'scheduled', or 'rejected'.
5. System updates the status of the appointment request.

User Personas

Patient Sarah

Role: 32, Marketing Manager

Needs & Goals: Quickly and easily request appointments online without phone calls; receive timely updates on appointment status.

Pain Points: Long phone wait times; difficulty scheduling during work hours; unclear appointment booking processes.

Clinic Staff Mark

Role: 45, Clinic Administrator

Needs & Goals: Efficiently manage incoming appointment requests; reduce manual data entry and phone calls; maintain an organized schedule for practitioners.

Pain Points: Overwhelmed by phone calls and voicemails; manual scheduling errors; difficulty tracking patient requests and communication.

Design

Principles

Simplicity: The interface should be intuitive and easy to navigate for all users, minimizing steps to complete tasks.
Efficiency: Enable quick and seamless appointment requests for patients and efficient management for clinic staff.
Clarity: Provide clear instructions, form fields, and feedback to users, ensuring transparency in the appointment process.
Accessibility: Design with inclusivity in mind, ensuring the application is usable by individuals with diverse needs.
Security: Implement robust security measures to protect sensitive patient and clinic data.

Sitemap

- Authentication (/) - The entry point for users to authenticate.
- Login/Signup (/auth) - User login and signup page.
- Appointment Request (/appointment) - Section for users to request and confirm appointments.
- Appointment Form (/appointment/form) - Form for users to submit appointment details.
- Confirmation (/appointment/confirmation) - Page confirming successful appointment request submission.
- Admin Dashboard (/admin) - Dashboard for clinic staff to manage appointment requests.
- View Requests (/admin/requests) - List of all submitted appointment requests.
- Request Details (/admin/requests/:id) - Detailed view of a single appointment request.

Navigation Design

Patient Navigation:
- **Login/Signup Page:** Clear calls to action for both login and signup.
- **Post-Login:** Direct navigation to the 'Appointment Request Form'.
- **Confirmation Page:** Simple message with no further navigation needed, perhaps a link back to a 'home' or 'dashboard' if more features were to be added.

Admin Staff Navigation:
- **Admin Login:** Separate login for clinic staff.
- **Admin Dashboard:** Primary navigation to 'View Requests'.
- **Request Details:** Navigation from 'View Requests' to individual 'Request Details' pages.

User Journeys

Sarah realizes she needs to see a doctor and prefers to book online.

Context: Patient Sarah needs to request an appointment with the clinic.

Steps:

1. **[User]** Navigates to the clinic's appointment app.
2. **[User]** Logs in with existing credentials or signs up for a new account.
3. **[System]** Displays the appointment request form.
4. **[User]** Fills in name, contact number, preferred date and time, and describes their problem.
5. **[User]** Reviews the entered information and clicks 'Submit Request'.
6. **[System]** Confirms successful submission and provides next steps (e.g., 'Clinic will contact you within 24 hours').

A new appointment request is submitted by a patient.

Context: Clinic Staff Mark needs to manage incoming appointment requests from patients.

Steps:

1. **[Staff]** Logs into the Admin Dashboard.
2. **[System]** Displays a list of new and pending appointment requests.
3. **[Staff]** Selects a request to view detailed information (patient name, contact, preferred time, problem description).
4. **[Staff]** Reviews the request and decides on a course of action (e.g., schedule, contact patient, reject).
5. **[Staff]** Updates the status of the request in the system (e.g., 'Reviewed', 'Scheduled', 'Rejected').
6. **[System]** Records the status update and potentially triggers an automated notification to the patient.

Sample Data

users

id	uid	name	email	appr…	prov…	last…	crea…	first…	upd…	sign…	profi…	encr…	allo…
1	user…	User 1	user…	true	email	Last1	2026…	First1	2026…	5	true	hash…	false
2	user…	User 2	user…	false	email	Last2	2026…	First2	2026…	10	false	hash…	false

organizations

id	name	created_at	updated_at
1	Organization 1	2026-03-06T13:29:08.066142+00:00	2026-03-06T13:29:08.066142+00:00
2	Organization 2	2026-03-06T13:29:08.066142+00:00	2026-03-06T13:29:08.066142+00:00

roles

id	name	created_at	updated_at
1	Role 1	2026-03-06T13:29:08.066150+00:00	2026-03-06T13:29:08.066150+00:00
2	Admin Role 2	2026-03-06T13:29:08.066150+00:00	2026-03-06T13:29:08.066150+00:00

users_roles

id	role_id	user_id	created_at	updated_at	organization_id
1	1	1	2026-03-06T13:29:0…	2026-03-06T13:29:0…	1
2	2	2	2026-03-06T13:29:0…	2026-03-06T13:29:0…	1

appointment_requests

name	status	user_id	reviewed	contacted	description	phone_n…	preferred…	preferred…	organiza…
Maria Go…	pending	1			Persisten…	+1-555-2…	2026-03-15	09:30	1
David Liu	pending	2			Mild ches…	+1-555-3…	2026-03-18	14:00	1

User Signup

MVP User Authentication

As a new user, **I want** to create an account, **so that** I can access the appointment request form

Acceptance Criteria:

- User can navigate to a signup page.
- User can enter an email address and password.
- User receives confirmation upon successful account creation.
- User is automatically logged in after signup.

User Login

MVP User Authentication

As a registered user, **I want** to log in to my account, **so that** I can submit an appointment request

Acceptance Criteria:

- User can navigate to a login page.
- User can enter their registered email address and password.
- System authenticates user credentials.
- User is redirected to the appointment request form upon successful login.
- System displays an error message for invalid credentials.

Fill Appointment Request Form

MVP Appointment Request Form

As a logged-in user, **I want** to request an appointment, **so that** I can schedule a visit to the clinic

Acceptance Criteria:

- User can access the appointment request form after logging in.
- Form includes fields for Name, Contact Number, Preferred Date, Preferred Time, and Problem Description.
- All required fields are clearly marked.
- User can select a preferred date from a calendar or date picker.
- User can select a preferred time from a time picker or dropdown.
- User can submit the form.

Secure Appointment Request Submission

MVP Backend API for Request Submission

As a system, **I want** to securely receive and store appointment requests, **so that** clinic staff can access and manage them

Acceptance Criteria:

- The backend API receives appointment request data securely.
- The API validates all incoming data fields (name, number, date, time, problem description).
- The API stores the validated appointment request in the database.
- The API returns a success confirmation to the frontend upon successful submission.
- The API handles errors gracefully and returns appropriate error messages for invalid submissions.

Manage Appointment Requests (Admin)

MVP Admin Dashboard for Request Management

As a clinic administrator, **I want** to view and manage submitted appointment requests, **so that** I can efficiently schedule and respond to patient inquiries

Acceptance Criteria:

- Clinic staff can log in to an admin dashboard.
- The dashboard displays a list of all submitted appointment requests.
- Each request shows the user's name, contact number, preferred date/time, and problem description.
- Staff can view details of individual appointment requests.
- Staff can mark requests as 'reviewed' or 'contacted'.
- The dashboard allows for basic filtering or sorting of requests (e.g., by date).

Scoping Estimation

Effort by Role Summary (Days)

Role	Traditional Method	Codalio Method	Savings
QA Engineer	10.50	10.50	0.00
UI/UX Designer	7.00	2.10	-4.90
Systems Engineer	5.00	1.50	-3.50
Backend Developer	12.50	5.00	-7.50
Frontend Developer	12.50	7.50	-5.00
Total	**47.50**	**26.60**	**-20.90**

Story Points Breakdown

User Signup

DESIGN	FRONTEND	BACKEND	QA	SYSTEM	TOTAL
3.0	5.0	6.0	4.0	2.0	20.0 Days

QA ENGINEER

- Develop test cases for signup flow
- Validate all input fields and error messages
- Test successful account creation and login
- Perform security testing for user registration

UI/UX DESIGNER

- Design signup form layout and elements
- Create responsive design for signup page
- Design success and error states

SYSTEMS ENGINEER

- Configure user database schema for new users
- Set up API gateway for signup endpoint
- Implement basic logging for signup events

BACKEND DEVELOPER

- Create user signup API endpoint
- Implement user registration logic and password hashing
- Generate and return authentication token (JWT)
- Store new user data in the database

FRONTEND DEVELOPER

- Build signup form component with input fields
- Implement client-side form validation
- Integrate with backend signup API
- Handle successful signup and automatic login

User Login

DESIGN	FRONTEND	BACKEND	QA	SYSTEM	TOTAL
2.0	4.0	5.0	3.0	2.0	16.0 Days

QA ENGINEER

- Develop test cases for login flow
- Validate correct and incorrect credentials
- Test error message display
- Perform security testing for login

UI/UX DESIGNER

- Design login form layout and elements
- Create responsive design for login page
- Design error states for invalid credentials

SYSTEMS ENGINEER

- Configure authentication service for token handling
- Set up secure session management
- Monitor login attempts for anomalies

BACKEND DEVELOPER

- Create user login API endpoint
- Implement user authentication logic
- Generate and return authentication token (JWT) upon successful login

FRONTEND DEVELOPER

- Build login form component
- Implement client-side form validation
- Integrate with backend login API
- Store authentication token and redirect user

Fill Appointment Request Form

DESIGN	FRONTEND	BACKEND	QA	SYSTEM	TOTAL
4.0	7.0	1.0	4.0	0.0	16.0 Days

QA ENGINEER

- Develop test cases for all form fields and validations
- Test date and time picker functionality
- Validate form submission readiness
- Test responsive behavior of the form

UI/UX DESIGNER

- Design appointment request form layout
- Create mockups for date and time pickers
- Define responsive behavior for the form
- Design validation feedback for form fields

SYSTEMS ENGINEER

- Ensure frontend deployment environment supports form components

BACKEND DEVELOPER

- Define API contract for appointment request submission (no implementation in this story)

FRONTEND DEVELOPER

- Build appointment request form component
- Implement input fields for name, contact, date, time, description
- Integrate date and time pickers
- Implement client-side form validation for all fields
- Prepare data for submission to backend API

Secure Appointment Request Submission

DESIGN	FRONTEND	BACKEND	QA	SYSTEM	TOTAL
1.0	2.0	7.0	5.0	3.0	18.0 Days

QA ENGINEER

- Develop test cases for API endpoint validation
- Test data persistence in the database
- Validate error handling for invalid submissions
- Perform security testing on the API endpoint

UI/UX DESIGNER

- Design confirmation message for successful submission
- Design error messages for submission failures

SYSTEMS ENGINEER

- Configure database for storing appointment requests
- Set up API security measures (e.g., rate limiting)
- Implement monitoring for API endpoint performance and errors

BACKEND DEVELOPER

- Create API endpoint for receiving appointment requests
- Implement server-side data validation for all incoming fields
- Store validated appointment request in the database
- Handle errors gracefully and return appropriate responses

FRONTEND DEVELOPER

- Integrate form submission with the new backend API endpoint
- Handle success and error responses from the API

Manage Appointment Requests (Admin)

DESIGN	FRONTEND	BACKEND	QA	SYSTEM	TOTAL
4.0	7.0	6.0	5.0	3.0	25.0 Days

QA ENGINEER

- Develop test cases for admin dashboard functionality
- Validate data display, filtering, and sorting
- Test status update functionality
- Verify admin access control and authorization

UI/UX DESIGNER

- Design admin dashboard layout and navigation
- Create mockups for request list and detail views
- Design filtering and sorting controls
- Define responsive behavior for the dashboard

SYSTEMS ENGINEER

- Configure admin user roles and permissions
- Set up secure access to the admin dashboard
- Implement monitoring for admin dashboard activity

BACKEND DEVELOPER

- Create API endpoints for fetching all appointment requests
- Implement API endpoint for updating request status (e.g., 'reviewed', 'contacted')
- Implement authentication and authorization for admin access

FRONTEND DEVELOPER

- Build admin dashboard UI to display appointment requests
- Implement table/list view for requests with details
- Add filtering and sorting functionality
- Integrate with backend API to fetch and update requests

Market Sizing

Total Addressable Market (TAM)

Estimated Value: $120,000,000,000.0

Total Addressable Market for the North American Digital Health Market, encompassing all healthcare technology solutions including electronic health records, telehealth, patient engagement, and practice management software.

Serviceable Available Market (SAM)

Estimated Value: $18,000,000,000.0

Serviceable Available Market for North American Patient Engagement and Practice Management Software, specifically focusing on solutions for appointment scheduling and patient communication for clinics.

Serviceable Obtainable Market (SOM)

Estimated Value: $450,000,000.0

Serviceable Obtainable Market, representing the realistic market share a new appointment booking platform can capture within the North American Patient Engagement and Practice Management Software Market over a 5-year horizon.

Key Assumptions

Customer type: Clinics (Clinic Administrators/Staff)

Time horizon years: 5

Pricing model basis: Subscription-based per clinic per month

Avg revenue per user per year: 900.0

Expected market penetration percent: 2.5

Industry category: Healthcare Technology

Geographic scope: North America

Industry growth rate cagr percent: 18.0

Key competitors: ["Zocdoc", "MyChart"]

Advantage summary: Seamless, user-friendly platform for patients; reduces administrative burden for clinics; improves patient access; intuitive and efficient appointment scheduling.